

LoginService

Relatorio de Seguranca - EHT

Itau - Avaliacao de Qualidade e Seguranca

Data: 24 de April de 2026

1. Sumario Executivo

Data/Hora do Teste:	2026-04-24 17:18:55
Servico Analisado:	LoginService - http://localhost:8081/api/v1
Tecnologia:	Spring Boot 4.0.5 Java 21 JWT Redis Spring Security
Total de Testes:	37
Testes Aprovados:	29
Vulnerabilidades Encontradas:	8
Taxa de Aprovacao:	78.4%

Classificacao de Risco das Vulnerabilidades

CRÍTICO	1 ocorrencia
ALTO	3 ocorrencias
MÉDIO	2 ocorrencias
BAIXO	2 ocorrencias

O servico LoginService apresenta uma base de seguranca razoavel, com proteções contra injeção SQL, XSS e configuração correta de sessão stateless via JWT. Contudo, foram identificadas vulnerabilidades criticas e altas que devem ser corrigidas antes de qualquer deploy em producao, especialmente a exposicao da symmetricKey no endpoint /me e o JWT secret hardcoded no codigo-fonte.

2. Detalhamento dos Testes por Categoria

Autenticação

PASS	INFO	Acesso /me sem token JWT
Resultado: HTTP 401 (esperado 401)		
PASS	INFO	Token JWT completamente inválido
Resultado: HTTP 401		
PASS	INFO	Token JWT expirado (forjado)

Resultado: HTTP 401		
PASS	INFO	Ataque JWT Algorithm 'none'
Resultado: HTTP 401		
PASS	INFO	Token sem prefixo 'Bearer'
Resultado: HTTP 401		

Injeção SQL

PASS	BAIXO	SQL Injection: ' OR '1'='1
Resultado: HTTP 401		
PASS	BAIXO	SQL Injection: ' OR 1=1--
Resultado: HTTP 401		
PASS	BAIXO	SQL Injection: admin'--
Resultado: HTTP 401		
PASS	BAIXO	SQL Injection: ' UNION SELECT 1,2,3--
Resultado: HTTP 401		
PASS	BAIXO	SQL Injection: '; DROP TABLE users;--
Resultado: HTTP 401		
PASS	BAIXO	SQL Injection na senha
Resultado: HTTP 401		

XSS

PASS	BAIXO	XSS payload: <script>alert(1)</script>
Resultado: HTTP 401		
PASS	BAIXO	XSS payload: javascript:alert(1)
Resultado: HTTP 401		
PASS	BAIXO	XSS payload:
Resultado: HTTP 401		

Validação

VULN	BAIXO	JSON malformado no body
Resultado: HTTP 401		
		Risco: AVISO: comportamento inesperado
PASS	BAIXO	Credenciais vazias
Resultado: HTTP 401		
PASS	BAIXO	Body JSON sem campos obrigatórios
Resultado: HTTP 401		

DoS Input

PASS	BAIXO	Payload extremamente grande (100KB username)
Resultado: HTTP 401		

Força Bruta

VULN	ALTO	Rate limiting após múltiplas tentativas incorretas
Resultado: Bloqueado: Não Último HTTP: 401 0.5s		
		Risco: ALTO: sem rate limiting - vulnerável a força bruta

Enumeração

PASS	BAIXO	Timing attack - diferença entre usuário existente/inexistente
Resultado: Existente: 0.004s Inexistente: 0.003s Diferença: 0.001s		
PASS	INFO	Código HTTP diferente para usuário existente vs inexistente
Resultado: Existente: HTTP 401 Inexistente: HTTP 401		

Headers HTTP

PASS	INFO	Header 'X-Content-Type-Options'
Resultado: Valor: nosniff		
PASS	INFO	Header 'X-Frame-Options'
Resultado: Valor: DENY		
VULN	ALTO	Header 'Strict-Transport-Security'
Resultado: Valor: AUSENTE		
Risco: Esperado: max-age=...		
VULN	MÉDIO	Header 'Content-Security-Policy'
Resultado: Valor: AUSENTE		
Risco: Esperado: presente		
PASS	INFO	Header 'X-XSS-Protection'
Resultado: Valor: 0		
PASS	INFO	Header 'Cache-Control'
Resultado: Valor: no-cache, no-store, max-age=0, must-revalidate		
VULN	BAIXO	Header 'Referrer-Policy'
Resultado: Valor: AUSENTE		
Risco: Esperado: presente		
PASS	INFO	Exposição de informações do servidor (Server / X-Powered-By)
Resultado: Server: " X-Powered-By: "		

Exposição de Dados

PASS	INFO	Stack trace exposto em respostas de erro
Resultado: Stack trace não exposto		

CSRF

VULN	MÉDIO	CSRF Protection desabilitada no SecurityConfig
Resultado: csrf.disable() detectado via análise estática		
Risco: MÉDIO: CSRF desabilitado - aceitável para APIs REST stateless com JWT, porém deve ser documentado		

CORS

PASS	INFO	CORS com Origin malicioso
Resultado: Access-Control-Allow-Origin: "		

Transporte

VULN	ALTO	HTTPS não habilitado (ambiente local sem TLS)
Resultado: Serviço rodando em HTTP na porta 8081		
Risco: ALTO: em produção DEVE usar HTTPS/TLS. Sem HTTPS, JWTs podem ser interceptados.		

Configuração

VULN	CRÍTICO	JWT secret hardcoded em application.yaml
Resultado: Secret: 'ChangeThisSecretKeyForProdUseAtLeast32Chars!'		
Risco: CRÍTICO: secret JWT hardcoded no fonte - usar variável de ambiente em produção		
PASS	BAIXO	H2 Console Web habilitado
Resultado: HTTP 404 no endpoint /h2-console		
PASS	INFO	Spring Actuator exposto sem autenticação
Resultado: HTTP 401		
PASS	INFO	Gerenciamento de sessão stateless (STATELESS)
Resultado: SessionCreationPolicy.STATELESS configurado		

3. Vulnerabilidades Criticas e Recomendacoes

CRÍTICO	JWT secret hardcoded em application.yaml Achado: Secret: 'ChangeThisSecretKeyForProdUseAtLeast32Chars!' <i>Recomendacao: Mover o segredo JWT para variavel de ambiente segura (ex: JWT_SECRET). Nunca commitar secrets no codigo-fonte. Usar AWS Secrets Manager, HashiCorp Vault ou variaveis de ambiente injetadas pelo orchestrador (Kubernetes Secrets).</i>
ALTO	Rate limiting após múltiplas tentativas incorretas Achado: Bloqueado: Não Último HTTP: 401 0.5s
ALTO	Header 'Strict-Transport-Security' Achado: Valor: AUSENTE
ALTO	HTTPS não habilitado (ambiente local sem TLS) Achado: Serviço rodando em HTTP na porta 8081
MÉDIO	Header 'Content-Security-Policy' Achado: Valor: AUSENTE
MÉDIO	CSRF Protection desabilitada no SecurityConfig Achado: csrf.disable() detectado via análise estática <i>Recomendacao: CSRF pode ser desabilitado para APIs REST puras que usam JWT (stateless). Porem, documentar explicitamente essa decisao e garantir que o token JWT nao seja armazenado em cookies (usar localStorage ou sessionStorage no cliente).</i>
BAIXO	JSON malformatado no body Achado: HTTP 401
BAIXO	Header 'Referrer-Policy' Achado: Valor: AUSENTE

4. Analise Estatica doCodigo-Fonte

Achados da Analise Estatica

CRÍTICO	JwtServiceImpl.java JWT secret derivado de String.getBytes() sem charset explicito. Pode gerar chave diferente em JVMs com default charset distinto.
ALTO	SessionUtils.java Uso de variaveis estaticas mutaveis (static SessionService, JwtService). Nao e thread-safe em cenarios de recarga de contexto Spring.
MÉDIO	LoginServiceImpl.java Chamada desnecessaria a userRepositoryPort.findByUsername() apos autenticacao bem-sucedida pelo AuthenticationManager (dupla consulta ao banco).
MÉDIO	DataInitializer.java Senha do usuario inicial hardcoded ('231299'). Deve ser configuravel via variavel de ambiente ou removida do DataInitializer em producao.
MÉDIO	application.yaml H2 console habilitado (h2.console.enabled: true) - nunca habilitar em producao.
BAIXO	SecurityConfig.java CORS nao configurado explicitamente. Adicionar CorsConfigurationSource para controle granular de origens permitidas.
BAIXO	SessionDTO.java SessionDTO retornado diretamente no /me expoe symmetricKey - criar DTO separado para a resposta publica.

5. Conclusao e Proximos Passos

O LoginService demonstra uma arquitetura bem estruturada com uso adequado de Spring Security, JWT e Redis para

gerenciamento de sessoes stateless. As vulnerabilidades criticas identificadas - especialmente a exposicao da symmetricKey e o JWT secret hardcoded - devem ser corrigidas com prioridade maxima antes de qualquer deploy em ambiente de producao ou staging.

Acoes Imediatas (Sprint Atual)

1. Remover symmetricKey do response do endpoint /me (criar MeResponseDTO)
2. Mover JWT secret para variavel de ambiente segura (nao commitar no git)
3. Implementar rate limiting no endpoint /auth/login (Bucket4j ou API Gateway)
4. Corrigir comportamento de retorno HTTP 401 vs 403

Acoes de Medio Prazo

5. Configurar HTTPS/TLS em producao (certificado SSL + HSTS header)
6. Adicionar Content-Security-Policy e Referrer-Policy headers
7. Remover H2 console e usuario inicial hardcoded em producao
8. Refatorar SessionUtils para eliminar estado estatico mutavel
9. Implementar auditoria de login (logs de tentativas falhas por IP)